

PROJECT AND TEAM INFORMATION

Project Title

Smart Emergency Response System

Student / Team Information

<i>Team Name:</i>	<i>ALPHA</i>
Team member 1 (Team Lead)	<i>Pandey, Sambhawi – 2510380216 2510380216@geu.ac.in</i> 
Team member 2	<i>Rawat, Atharv – 2510370380 2510370380@geu.ac.in</i> 
Team member 3	<i>Lalotra, Aayush – 2510012608 2510012608@geu.ac.in</i> 
Team member 4	<i>Garg, Shivam – 2510386512 2510386512@geu.ac.in</i> 

PROPOSAL DESCRIPTION

Motivation

Emergency situations such as road accidents, fires, and medical emergencies require quick and efficient response. In a city, multiple emergencies may occur at the same time while only a limited number of ambulances, emergency vehicles, and response teams are available. In such situations, deciding which emergency should be handled first, which resource should be assigned, and which route should be followed can become difficult if the decisions are made manually.

The proposed Smart Emergency Response System aims to address this problem by developing an algorithm-based system that assists in prioritizing emergencies and allocating available emergency resources efficiently. The project will represent the city road network using a graph, where locations are represented as vertices and roads as weighted edges. Dijkstra's shortest-path algorithm will be used to determine an efficient route between locations.

A priority queue will be used to handle emergencies according to their severity, ensuring that critical cases receive higher priority. Queues, searching, sorting, linked lists/vectors, and hashing can also be incorporated for efficient data management.

The project will be implemented in C++ using Object-Oriented Programming, with classes representing entities such as emergencies, ambulances, hospitals, and locations. The initial system will be a simulation-based prototype designed to demonstrate how DSA and OOP can be applied to a real-world problem.

State of the Art / Current solution

Currently, emergency response generally involves emergency call centers, dispatchers, ambulance services, police, fire departments, and hospitals working together to respond to incidents. Emergency information is received, the incident is assessed, and an available response unit is dispatched to the location.

Modern emergency-management solutions may also use digital maps, GPS, communication systems, and location-based services to support dispatch and navigation. However, the effectiveness of response can still depend on the availability of resources, traffic conditions, communication between agencies, and the decisions made during dispatch.

The proposed project does not attempt to replace existing emergency services. Instead, it focuses on the algorithmic decision-making aspect of emergency response. The system will provide a simulation in which emergencies can be prioritized, available resources can be identified, and routes can be calculated using DSA.

The key gap addressed by the project is the development of an integrated educational prototype that demonstrates how priority queues, graphs, shortest-path algorithms, searching, sorting, and OOP can work together to support emergency-response decisions.

Project Goals and Milestones

The main goal of the project is to develop a C++-based Smart Emergency Response System that efficiently manages emergency requests, prioritizes incidents, allocates available emergency resources, and determines suitable routes using DSA and OOP concepts

Major Goals :

1. *Develop an emergency-reporting and management module.*
2. *Categorize emergencies according to severity.*
3. *Implement a priority queue/heap for emergency prioritization.*
4. *Represent the city road network using a graph.*
5. *Implement Dijkstra's algorithm for shortest-path calculation.*
6. *Manage ambulances and other emergency resources.*
7. *Use appropriate searching and sorting techniques for efficient data management.*
8. *Apply OOP concepts through modular classes and objects.*
9. *Build a menu-driven C++ prototype.*
10. *Test the system using different emergency and resource scenarios.*

Initial Milestones

Milestone 1: Requirement analysis and system design

Milestone 2: Design classes for Emergency, Ambulance, Hospital, Location, and Response System

Milestone 3: Implement emergency management using queues/priority queues

Milestone 4: Implement city representation using graphs

Milestone 5: Implement Dijkstra's shortest-path algorithm

Milestone 6: Integrate resource allocation and route selection

Milestone 7: Develop the complete menu-driven C++ prototype

Milestone 8: Testing, debugging, performance analysis, and final documentation

Project Approach

The project will be developed using a modular and incremental approach. First, the requirements of an emergency-response system will be analyzed and the major entities and operations will be identified. The system will then be designed using Object-Oriented Programming in C++.

Classes such as Emergency, Ambulance, Hospital, Location, and EmergencyResponseSystem will be created to represent the major components. Encapsulation will be used to protect data, while appropriate OOP principles will be applied to make the system modular and maintainable.

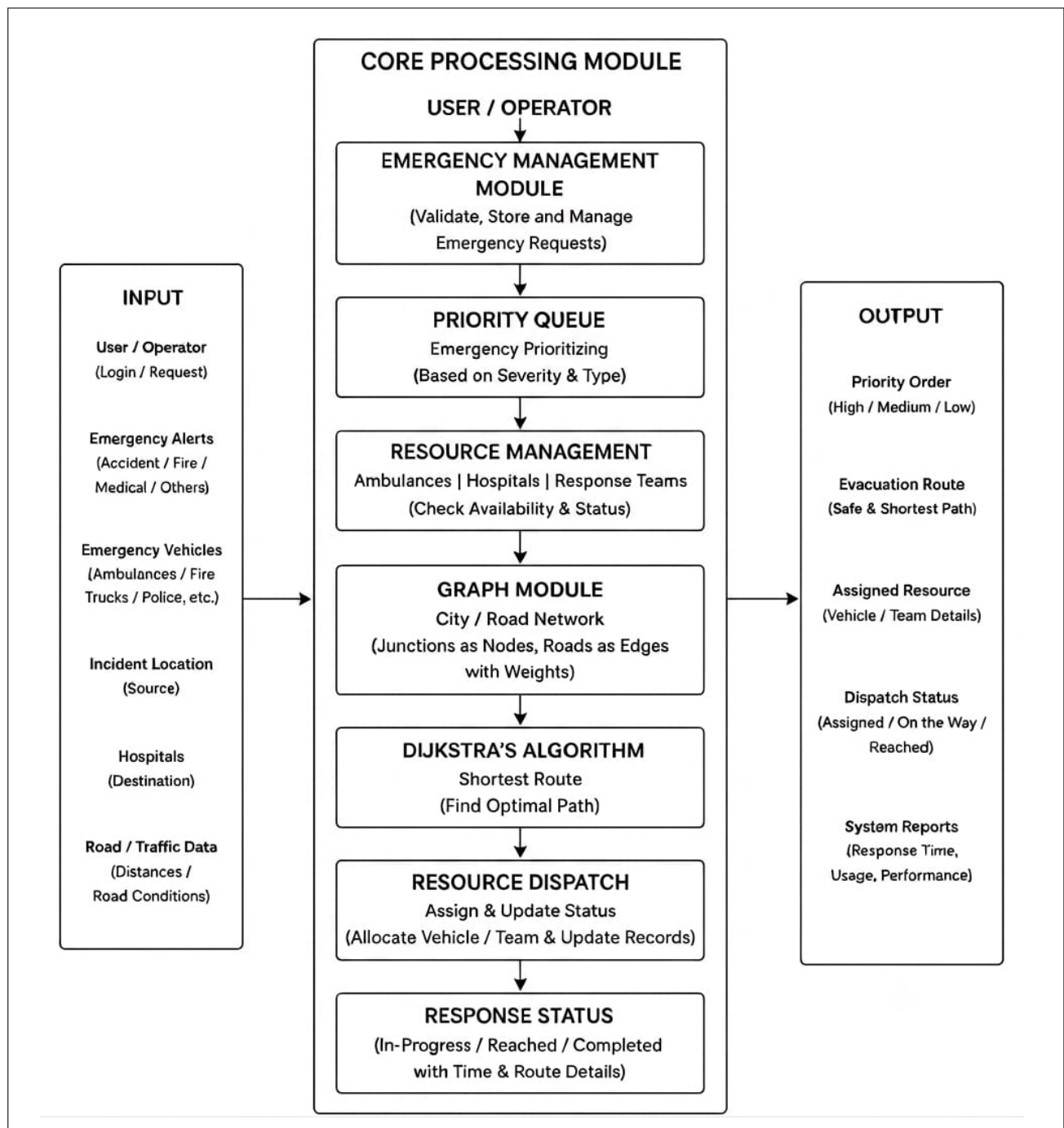
Different DSA concepts will be selected according to their practical use. A queue will manage incoming emergency requests, while a priority queue/heap will prioritize emergencies according to severity. The city will be represented as a weighted graph, with locations as vertices and roads as edges. Dijkstra's algorithm will calculate the shortest route between the emergency location and available response resources.

Searching and sorting techniques will be used to locate and organize emergency resources. Linked lists/vectors may be used for dynamic collections, while hashing can provide faster lookup of resources using unique IDs.

The initial platform will be a C++ console-based application developed and tested using a suitable C++ development environment such as Visual Studio Code. The system will use simulated data for emergencies, roads, ambulances, and hospitals.

Development will be incremental: individual data structures and algorithms will first be tested separately and then integrated into the complete emergency-response workflow.

System Architecture (High Level Diagram)



Project Outcome / Deliverables

The main outcome of the project will be a working C++ console-based Smart Emergency Response System demonstrating the practical application of DSA and OOP concepts to emergency management.

The system will provide the following functionalities:

- 1. Registration and management of emergency requests.*
- 2. Classification and prioritization of emergencies.*
- 3. Management of available ambulances and other resources.*
- 4. Representation of a city road network using a weighted graph.*
- 5. Shortest-route calculation using Dijkstra's algorithm.*
- 6. Selection and assignment of an available emergency resource.*
- 7. Updating resource and emergency status.*
- 8. Display of emergency, resource, and route information.*
- 9. Efficient data organization using appropriate searching and sorting techniques.*

Expected Deliverables :

- 1. C++ source code.*
- 2. Working menu-driven prototype.*
- 3. System architecture diagram.*
- 4. Test cases and results.*
- 5. Project report.*
- 6. Presentation/PPT and demonstration*

Assumptions

The initial system will operate as a simulation using predefined city locations, roads, emergency requests, ambulances, and hospitals. Road distances or travel costs will be represented using numerical weights. Ambulance and hospital availability will be provided as simulated data rather than obtained from real emergency departments. The system assumes that emergency severity can be represented using predefined priority levels. Real-time GPS, traffic information, external APIs, and live hospital data are outside the initial third-semester scope and may be considered for future development.

References

- 1. C++ Reference — cppreference.com*
- 2. Dijkstra's Algorithm — [CP-Algorithms](#)*
- 3. Class notes and reference material provided by the faculty*